LAPORAN PRAKTIKUM

TEKNOLOGI IOT II

Oleh :

Nama : LILI CAHYA PUTRI

NPM : 25782071

Kelas : TRI 3C

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026

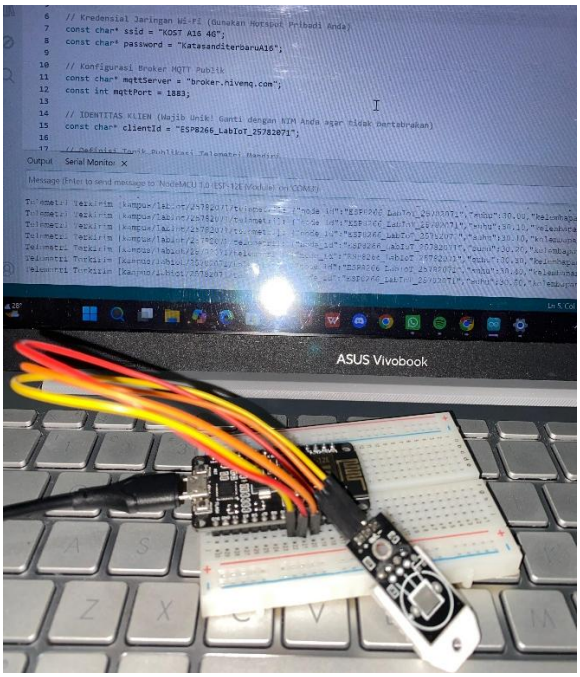
6.5 Praktikum

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Publisher*. Perangkat membaca data suhu dan kelembapan dari sensor DHT11/DHT22, mengemas parameter tersebut ke dalam dokumen JSON menggunakan ArduinoJson, lalu memublikasikannya setiap 5 detik ke broker publik broker.hivemq.com.

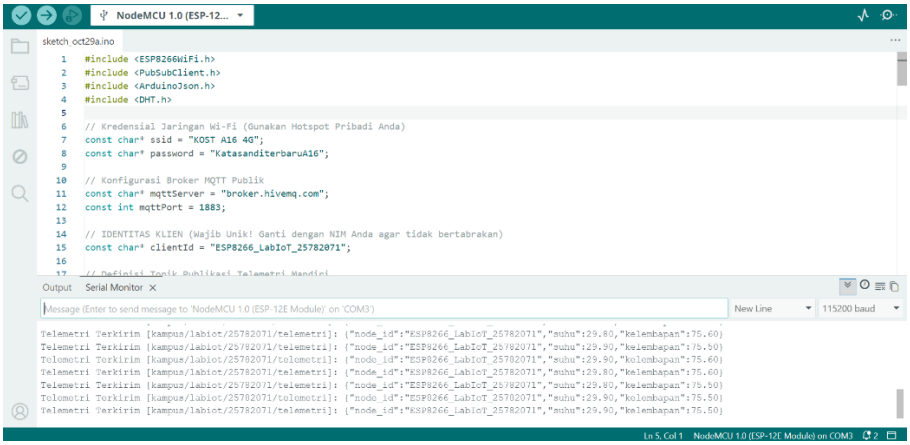
Langkah Kerja Rangkaian:

1. Pasang NodeMCU ESP8266 pada *breadboard*.
2. **Sensor DHT11/DHT22:** Hubungkan pin **VCC** ke **3V3**, pin **GND** ke **GND**, dan pin **Data** ke **D4 (GPIO 2)**.



Langkah Kerja Pemrograman:

1. Buka Arduino IDE, buat berkas sketsa baru (*File > New Sketch*).
2. Tuliskan kode program
3. Ganti NAMA_WIFI_ANDA, PASSWORD_WIFI_ANDA, clientId (gunakan NIM Anda), dan topik kampus/labiot/nim_anda/telemetri dengan NIM pribadi Anda.
4. Unggah (*upload*) program ke NodeMCU ESP8266, buka Serial Monitor pada *baud rate* **115200**.
5. Amati log: ESP8266 akan terhubung ke Wi-Fi, melakukan jabat tangan dengan broker broker.hivemq.com, dan memublikasikan paket JSON setiap 5 detik.



6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

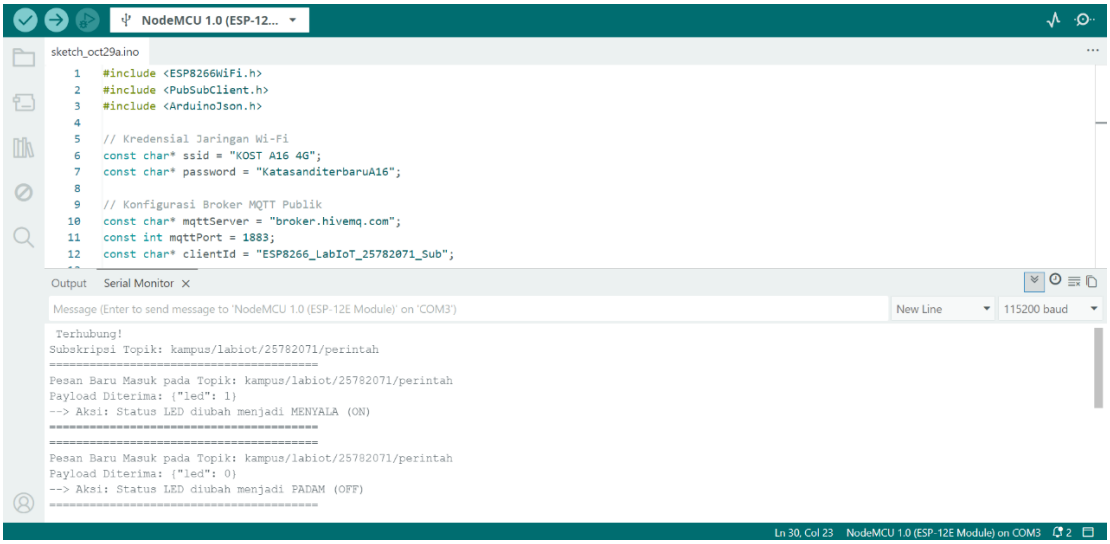
Pada praktikum ini, NodeMCU ESP8266 diprogram sebagai *Subscriber*. Mikrokontroler mendaftarkan minatnya pada topik perintah (kampus/labiot/nim_anda/perintah). Saat pengguna luar (melalui aplikasi penguji MQTT) memublikasikan perintah berformat JSON, fungsi *callback* NodeMCU akan membedah payload dan menyalakan/mematikan lampu LED indikator secara seketika.

Langkah Kerja Rangkaian:

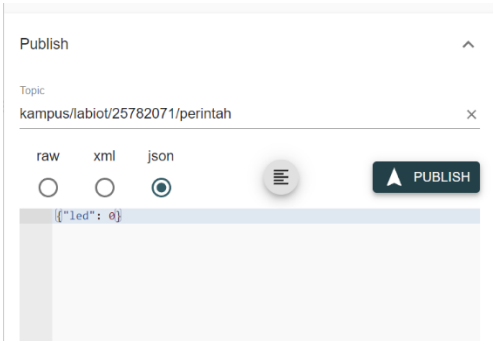
1. Pertahankan sensor DHT pada pin D4 dari Praktikum 1.
2. **LED Indikator:** Hubungkan kaki Anoda (kaki panjang) LED ke pin **D1 (GPIO 5)**, dan hubungkan kaki Katoda (kaki pendek) ke Resistor 220 Ωyang menuju ke **GND**.

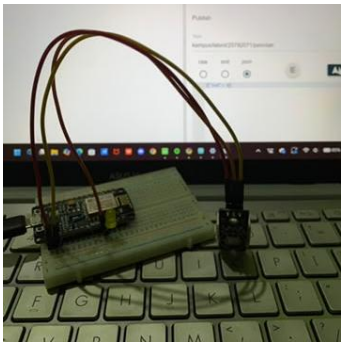
Langkah Kerja Pemrograman:

1. Buat sketsa baru di Arduino IDE, lalu masukkan kode
2. Unggah kode ke board NodeMCU Anda.

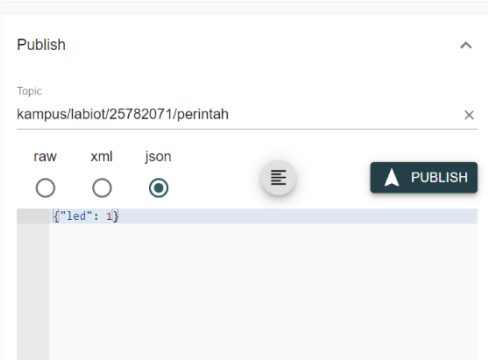


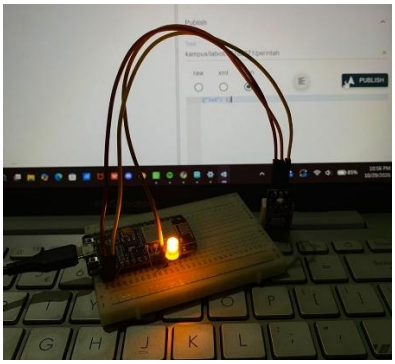
Saat kirim pesan/payload untuk **mematikan LED**: {"led": 0}





Saat kirim pesan/payload untuk **menyalakan LED**: {"led": 1}





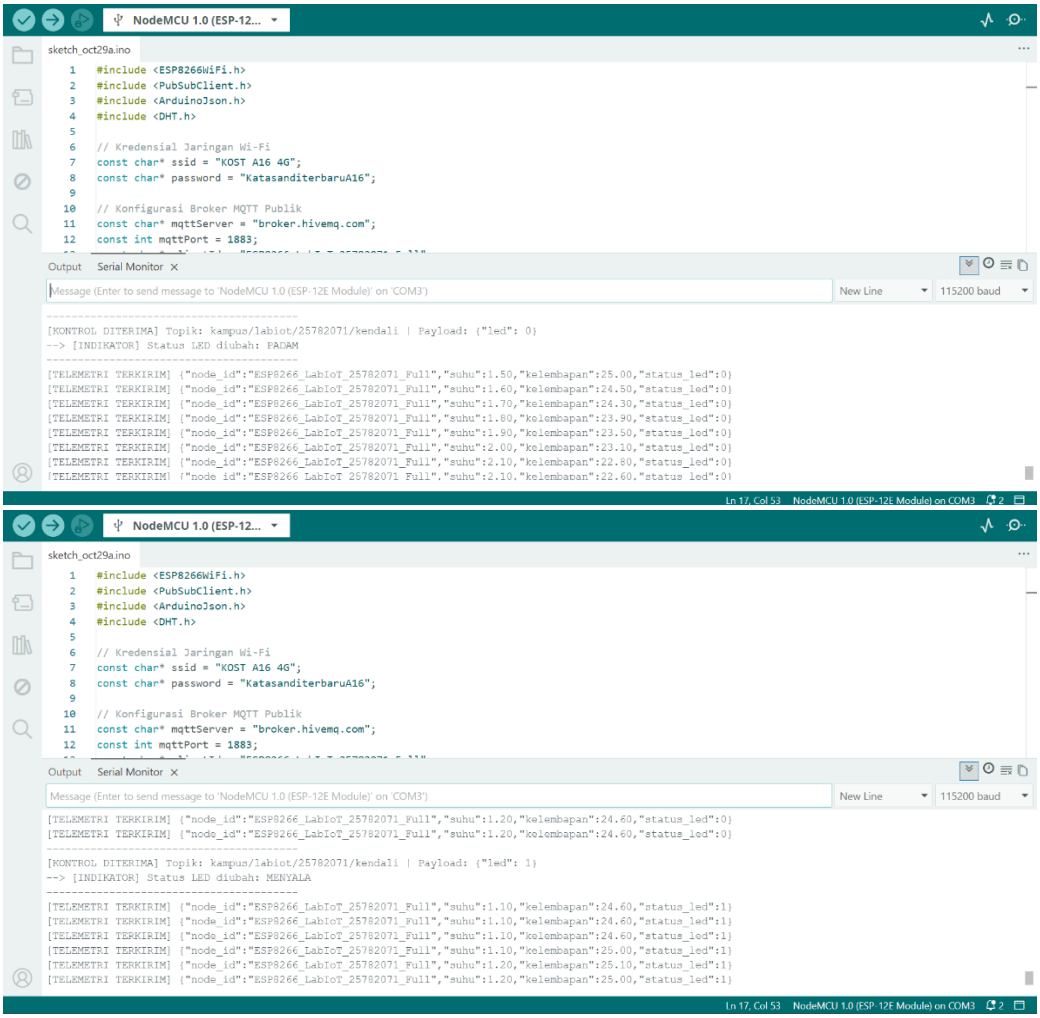
6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Pada praktikum ini, kita menyatukan kapabilitas Praktikum 1 dan Praktikum 2 ke dalam sebuah sistem terpadu (*Full-Duplex Telemetry & Control*). NodeMCU secara kontinu memublikasikan data telemetri sensor lingkungan (Suhu, Kelembapan, dan Status LED fisik) ke Broker setiap 4 detik, sembari tetap responsif mendengarkan dan mengeksekusi instruksi kontrol LED dari topik perintah kapan saja.

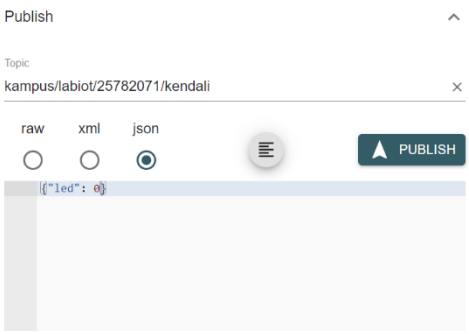
Langkah Kerja Pemrograman:

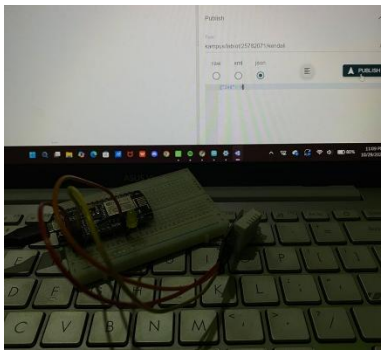
1. Masukkan kode terpadu ke Arduino IDE:
2. Unggah kode ke NodeMCU ESP8266 dan biarkan sistem berjalan.

laporan telemetri secara *real-time* bahwa LED akan merespons nyala/padam

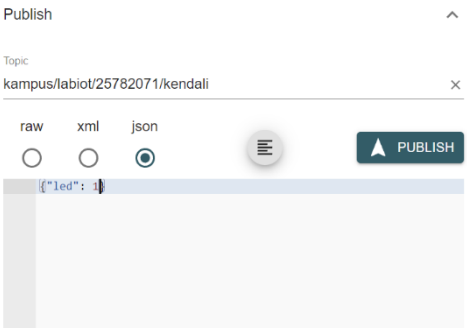


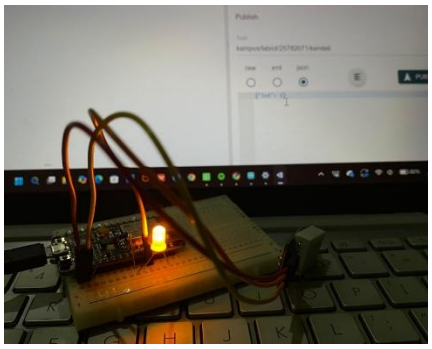
Saat kirim pesan/payload untuk **mematikan LED**: {"led": 0}



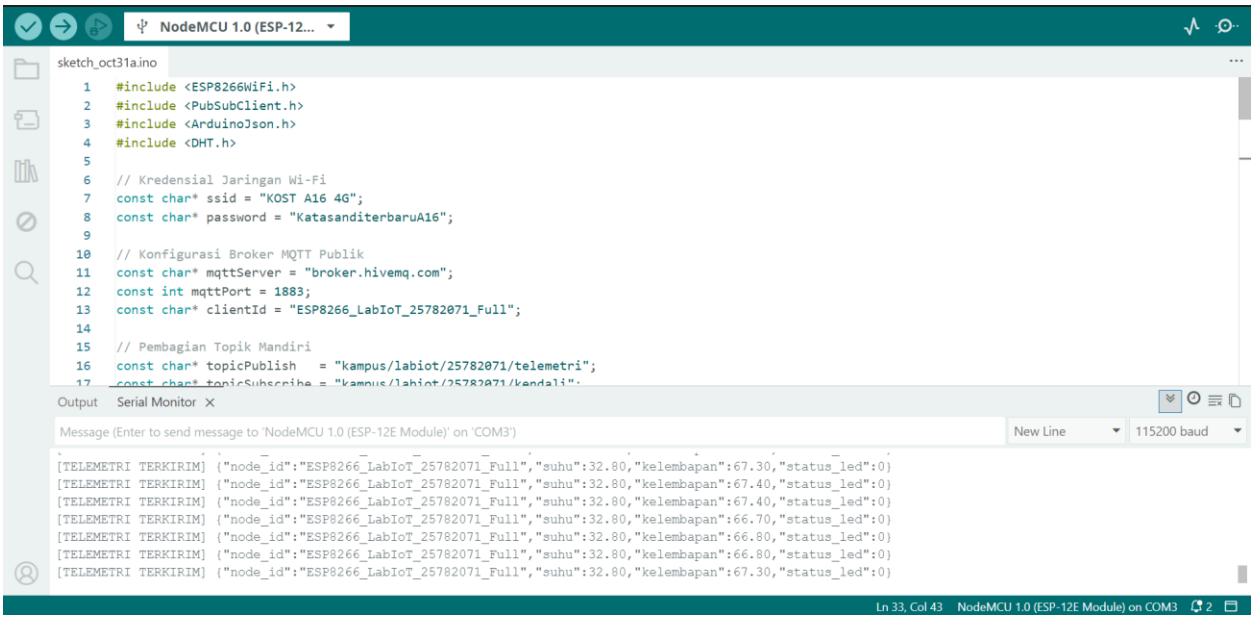


Saat kirim pesan/payload untuk **menyalakan LED**: {"led": 1}





6.5.4 Panduan Pengujian Diagnostik dengan MQTT Explorer



Berdasarkan tangkapan layar *Serial Monitor* Arduino IDE milikmu, program yang menggunakan pustaka seperti ESP8266WiFi.h, PubSubClient.h, ArduinoJson.h, dan DHT.h telah berhasil berjalan dengan baik. Perangkat NodeMCU ESP8266 sukses terhubung ke jaringan Wi-Fi KOST A16 4G dan terhubung ke MQTT Broker publik broker.hivemq.com melalui port 1883 dengan *ClientID* ESP8266_LabIoT_25782071_Full. Dalam pengujian ini, topik publikasi telemetri diatur pada jalur kampus/labiot/25782071/telemetri dan topik kendali pada kampus/labiot/25782071/kendali. Berdasarkan log yang tampil secara kontinu di *Serial Monitor*, sistem terbukti berhasil mengirimkan data berformat JSON berupa pembacaan sensor suhu sebesar 32.80°C, kelembapan sebesar 67.30%, serta status LED mati (status_led: 0) tanpa adanya kendala atau pesan *error*.

6.6 Latihan

Latihan 1: Analisis Arsitektur & Perbandingan Protokol

1. **Bandwidth:** MQTT jauh lebih hemat kuota/data karena header-nya sangat kecil (minimal 2 byte), sedangkan HTTP boros karena bawa header teks yang besar di tiap *request*-nya.
2. **Beban CPU:** MQTT lebih ringan karena pakai sistem *publish/subscribe* asinkron tanpa harus buka-tutup koneksi TCP terus-menerus seperti HTTP yang sifatnya sinkron.
3. **Distribusi 1-to-N:** MQTT gampang banget bagiin data ke banyak perangkat sekaligus lewat satu broker, sementara HTTP susah karena harus layani klien satu-satu secara manual.

Latihan 2: Analisis Hierarki Topik & Karakter Wildcard

1. pabrik/+/+/suhu (Pakai tanda + buat nyari semua lini dan mesin secara fleksibel).
2. pabrik/line2/m1/# (Pakai tanda # di belakang buat narik semua jenis sensor di mesin m1).
3. pabrik/# (Pakai # di root buat mantau semua data pabrik tanpa terkecuali).

Latihan 3: Implementasi Serialisasi Data Dinamis

Tambahan baris kode C++ di bagian pembentukan JSON:

```
C++

unsigned long uptime_detik = millis() / 1000;

String ip_lokal = WiFi.localIP().toString();


StaticJsonDocument<300> doc;

doc["suhu"] = suhu;

doc["kelembapan"] = kelembapan;

doc["status_led"] = status_led_sekarang;

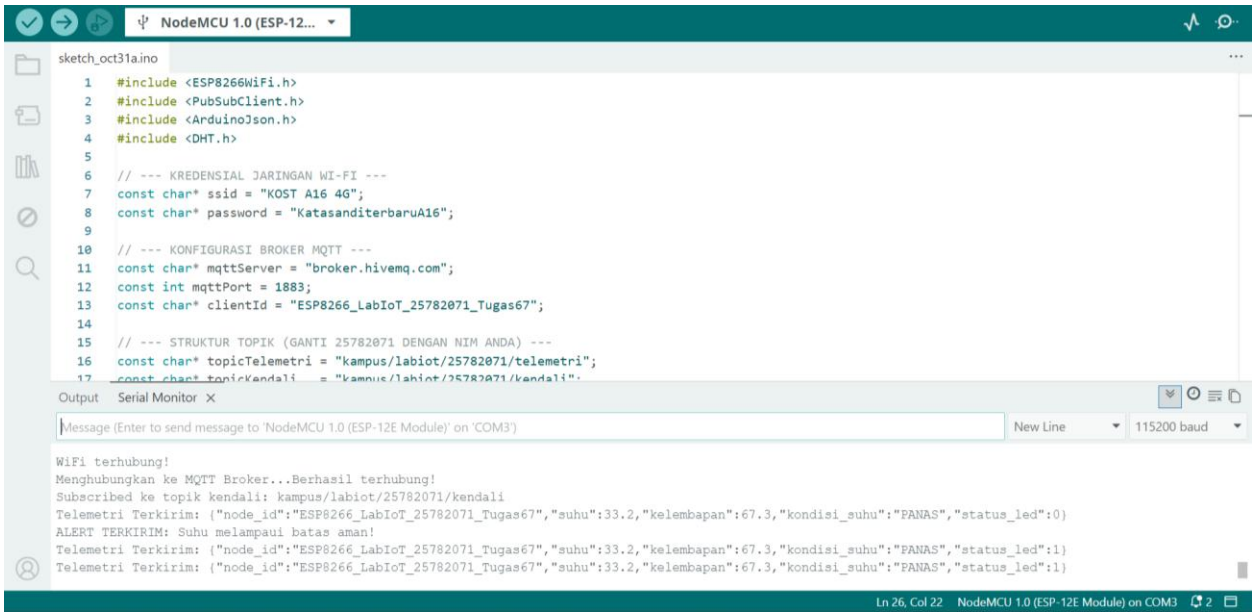
doc["uptime_detik"] = uptime_detik;   // Tambahan parameter uptime

doc["ip_lokal"] = ip_lokal;           // Tambahan parameter IP lokal
```

Latihan 4: Analisis Mekanisme Reconnect & Loop Blocking

1. **Akibat delay(6000) di loop():** Mikrokontroler bakal *blocking* (macet total) selama 6 detik. Akibatnya, perintah kendali dari MQTT broker bakal telat direspons dan perangkat bisa terputus (*disconnect*) karena gagal kirim sinyal *ping*.
2. **Cara kerja reconnect():** Fungsi ini ngecek kondisi !client.connected(). Kalau putus, sistem bakal otomatis nyambungin ulang ke broker dan langsung *subscribe* ulang ke topik kendali biar perangkat bisa normal lagi

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri



Berdasarkan hasil pengujian fungsionalitas sistem pada Tugas 6.7, perangkat NodeMCU ESP8266 berhasil melakukan inisialisasi koneksi Wi-Fi dan terhubung ke MQTT Broker dengan sukses, serta otomatis melakukan *subscribe* ke topik kendali. Ketika sistem beroperasi, sensor berhasil mencatat suhu lingkungan sebesar **33.2°C** dan kelembapan **67.3%**. Karena suhu tersebut melampaui ambang batas aman ($\leq$ 31.0°C), fitur *Edge Safety Engine* secara mandiri memicu pesan ALERT TERKIRIM: Suhu melampaui batas aman! ke kanal peringatan khusus (alert), mengubah parameter kondisi menjadi "PANAS", dan langsung mengaktifkan LED indikator ("status_led":1) tanpa menunggu perintah dari operator luar. Hal ini membuktikan bahwa mekanisme proteksi otomatis, serialisasi JSON dinamis, dan sistem kendali *closed-runner* pada industri telah berjalan dengan sangat responsif dan sesuai dengan rancangan tugas.